Exact 1-NN at Scale

GPU-Accelerated Forest Cover Classification

581,012 observations • 54-dimensional features • 7 forest classes

CPU naive/optimized → CUDA baseline/optimized
Coalesced memory / shared query cache / two-stage argmin

What are we solving?

INPUT

Reference landscape records: 572,820

Held-out test queries: 8,192

Each record contains 54 numeric features.

OUTPUT

For each query: find the single closest reference in feature space.

Predict its forest-cover class (1–7).

Why CUDA? Every query must compare with many candidate references.

A genuine, public large dataset

581,012
LANDSCAPE SAMPLES

54
INPUT FEATURES

7
COVER TYPES

SPLIT + LEAKAGE CONTROL

10 quantitative features standardized using reference-only statistics. 44 binary features preserved; class label excluded from distance.

Source: Blackard (1998), UCI Covertype • doi:10.24432/C50K5N • CC BY 4.0

Exact 1-nearest-neighbor (1-NN)

$$d^2(q, x_j) = \Sigma_{(f=1\ldots54)} (q^f - x_j^f)^2$$

ARGMIN

Find j* with minimum d^2. Return the class label of reference j*.

EXACT, NOT APPROXIMATE

No learned model or approximate index; output is a deterministic exhaustive nearest neighbor.

No square root needed — minimizing d and minimizing d^2 produce the same winner.

Serial CPU: two fair baselines

```
for each query q:
    best_d2 = +INF; best_idx = -1
    for each reference x[j]:
        sum = 0
        for f in 0..53:
            sum += (q[f]-x[j][f])²
            if sum > best_d2: break
        if sum < best_d2: update best
```

CPU NAIVE

Scan all references and all 54 features. Reference for correctness and overhead.

CPU EARLY ABANDON

Stop when partial squared sum > best. Exact but branching can be slower on small inputs.

CUDA baseline: each thread owns a candidate

GRID Y
One row of blocks per query

→

GRID X
256 candidate refs per block

→

PARTIAL
Each block writes (min d^2, index)

Final CUDA kernel reduces block winners → exact global nearest neighbor.

Baseline disadvantage: row-major AoS produces wide-stride global reads across a warp.

Optimization #1: coalesced SoA access

AoS • BASELINE

ref0: f0 f1 f2 f3 … f53
ref1: f0 f1 f2 f3 … f53
ref2: f0 f1 f2 f3 … f53

Adjacent threads access far-apart addresses.

SoA • OPTIMIZED

f0: ref0 ref1 ref2 ref3 …
f1: ref0 ref1 ref2 ref3 …
f2: ref0 ref1 ref2 ref3 …

Adjacent threads load contiguous references.

Tradeoff: transpose reference matrix once; amortize cost over many queries.

Optimization #2–4: reduce overhead

SHARED MEMORY

Each block cooperatively caches 54 query features. All threads reuse the same query.

REGISTER TILING

4 reference candidates per thread; 4× fewer partial block winners than AoS baseline.

WARP REDUCTION

Use shuffle instructions for min(distance, index); no global atomics.

Optimizations improve memory efficiency and overhead without changing the exact answer.

Work–Step analysis & APOD development

WORK: W = O(Q × N × D)

Q queries, N references, D features. Exhaustive CPU/CUDA both do O(QND) worst-case arithmetic.

IDEALIZED PARALLEL STEPS

Per query: O(RD + log B + ceil(T/B) + log B); T = ceil(N/(BR)). Excludes device scheduling.

APOD: ASSESS → PARALLELIZE → OPTIMIZE → DEPLOY

Measure CPU hotspot → map query/reference pairs to GPU → SoA, shared cache, register tiling and reduction → validate + benchmark.

B=256 threads/block; R=4 references/thread (optimized). Theory ≠ measured GPU milliseconds.

Why the parallel algorithm is correct

PARTITIONED MINIMUM

min over all references = min of per-block minima.

CONSISTENT TIES

Compare (distance^2, row ID) lexicographically.

SERIAL ORACLE TEST

Handmade fixture expects indices [0, 3, 4]. Independent randomized NumPy CPU test passes.

CUDA VALIDATION

Compare GPU answers with serial answers for every measured query; mismatches flagged.

How performance is measured

Q = 32	Q = 128	Q = 512	Q = 2048
SMALL	MEDIUM	LARGE	LARGER

SEARCH-ONLY

CPU early-abandon search wall time

GPU two-kernel elapsed time (CUDA events, after warm-up)

END-TO-END

GPU AoS→SoA + H2D + one search + D2H

Speedup = CPU time / GPU time

Measured results • fill after GPU run

Results must come from your own NVIDIA CUDA run

QUERIES	CPU ms	CUDA ms	E2E ms	SPEEDUP	MISMATCHES
32	—	—	—	—	—
128	—	—	—	—	—
512	—	—	—	—	—
2048	—	—	—	—	—

GPU results: NOT MEASURED. Run on actual NVIDIA GPU and replace placeholders before submission.

Live demo: reproducible commands

```
python -m pip install -r requirements.txt

python scripts/prepare_data.py

make cpu && make gpu

./build/knn_cuda --queries 32 --gpu-baseline --cpu-naive

python scripts/benchmark.py --queries 32 128 512 2048 --gpu-baseline
```

Windows: compile using nvcc in Developer PowerShell; use build/knn_cuda.exe.

What could limit speedup?

SMALL WORKLOADS

Kernel launch, transfer and layout-conversion overhead.

GPU VRAM

Reference tensor and intermediate block winners consume memory.

CPU PRUNING

Early abandon can reduce CPU operations well below the worst case.

NOT A GLOBAL OPTIMUM

Index-based and approximate searches solve different tradeoffs.

Takeaways & references

PROJECT DELIVERABLE

Reproducible CPU/CUDA implementation + exact agreement checks + hardware-dependent measurements.

MAIN IDEA

Expose one-to-many comparisons; optimize coalescing and reduction, not just thread count.

UCI Covertype — https://doi.org/10.24432/C50K5N

NVIDIA — CUDA C++ Programming Guide / Best Practices Guide

THANK YOU / QUESTIONS?